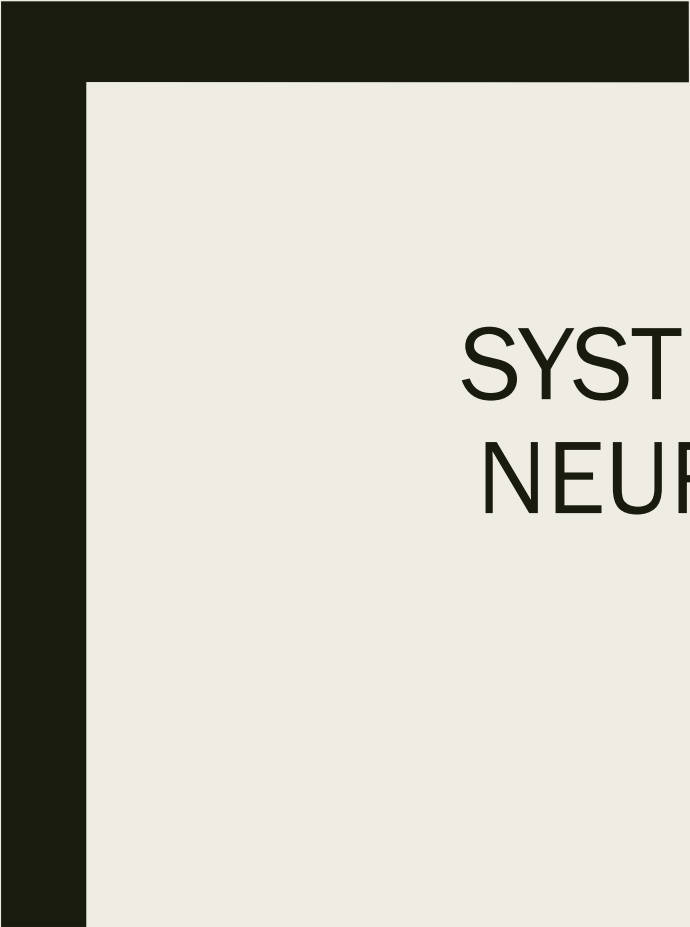
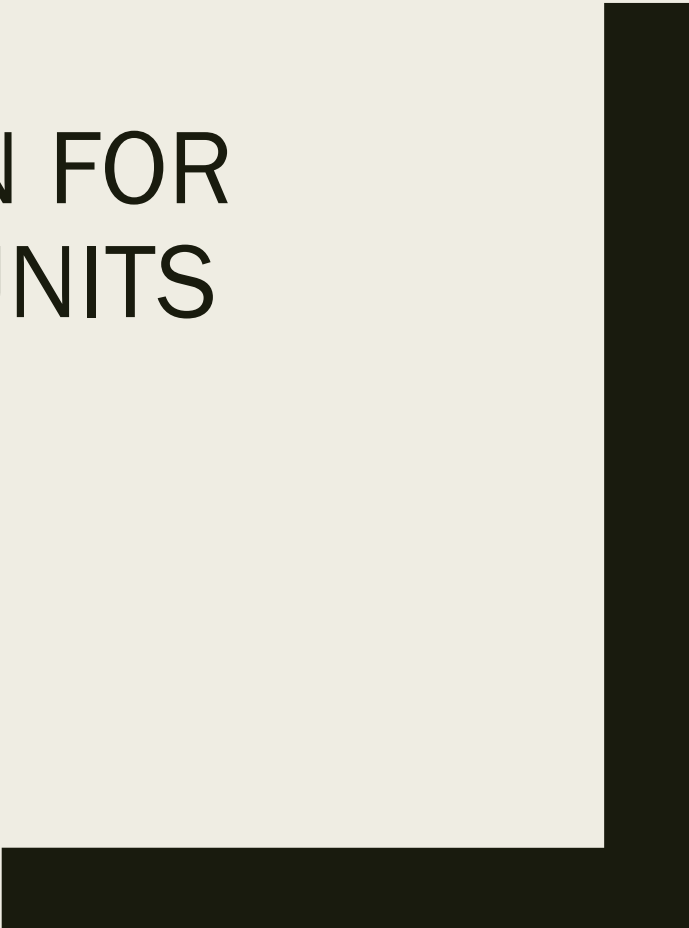




SYSTEM VIRTUALIZATION FOR NEURAL PROCESSING UNITS

논문 리뷰

26/01/14 세미나



목차

- 1. Abstract
- 2. Introduction
- 3. Background and Motivation
- 4. Towards NPU Virtualization
- 5. Conclusion

Abstract

Abstract

- 현대 클라우드 플랫폼은 NPU(신경망 처리 장치)와 같은 하드웨어 가속기를 채택해 왔다
- 시스템 가상화 지원의 부족으로 인해, 현재 클라우드 플랫폼에서 NPU를 사용하는 방식은 리소스 활용도가 낮거나 다중 사용자(multi-tenant) 애플리케이션 서비스 간의 격리가 제대로 이루어지지 않는 문제점을 안고 있다
- 이에 전체 소프트웨어 및 하드웨어 스택에 걸친 NPU 시스템 가상화 기술을 조사하고, NeuCloud라는 이름의 NPU 가상화 솔루션을 제시(미세 조정된 NPU 가상화 및 리소스 관리를 가능하게 하는 vNPU라는 유연한 NPU 추상화 모델을 제안)

Introduction

Introduction

- 현대 클라우드 플랫폼은 NPU(신경망 처리 장치)와 같은 하드웨어 가속기를 채택해 왔다
- NPU(신경망 처리 장치): 딥러닝과 같은 AI 작업을 효율적으로 수행하기 위해 설계된 전용 하드웨어 가속기
- 주요 역할
 1. AI 연산 가속
 2. 효율성
- 각 NPU 코어에는 Sas/Vus 포함됨

Introduction

- NPU를 사용하는 가장 일반적인 방법은 PCIe 패스스루(pass-through) 기술을 통해 단일 가상 머신 또는 컨테이너에 NPU 보드 전체를 독점적으로 할당
-> 심각한 리소스 저활용(SA/VU에 대한 불균형 수요)
- NPU의 컴퓨팅 활용도를 높이기 위해 현대 클라우드 플랫폼은 NPU에 대한 제한적인 가상화 지원을 구현
but 여전히 심각한 리소스 저활용 문제와 여러 문제 발생

Introduction

- 이에 NPU용 시스템 가상화 솔루션인 NewCloud 제안
- 세밀한 SA/VU 할당을 지원하기 위해 vNPU라는 추상화 모델
+ 다양한 SA 및 VU 수요를 가진 워크로드에 최적화된 vNPU 구성을 식별하는 알고리즘을 제안
- 여러 물리적 NPU 코어에 서로 다른 vNPU 인스턴스를 동적으로 할당할 수 있는 효율적인 매핑 정책을 설계하여 세밀한 공간 공유를 가능하게 함
- 컴퓨팅 유닛과 메모리 주소 공간의 분할을 통해 vNPU의 보안 및 성능 격리를 제공

Background and Motivation

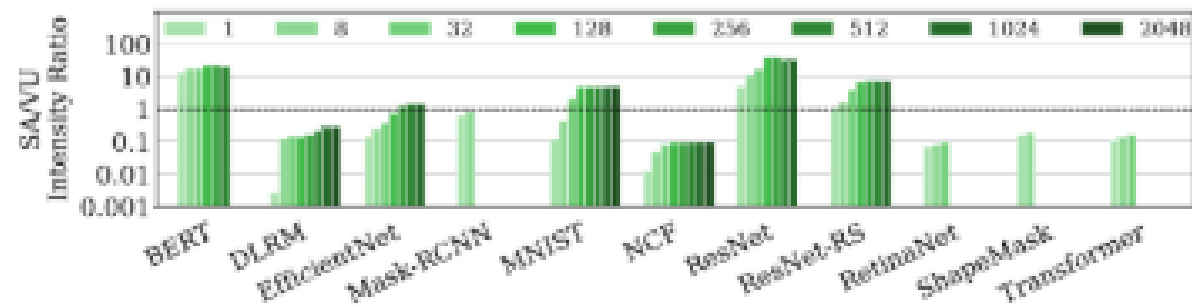
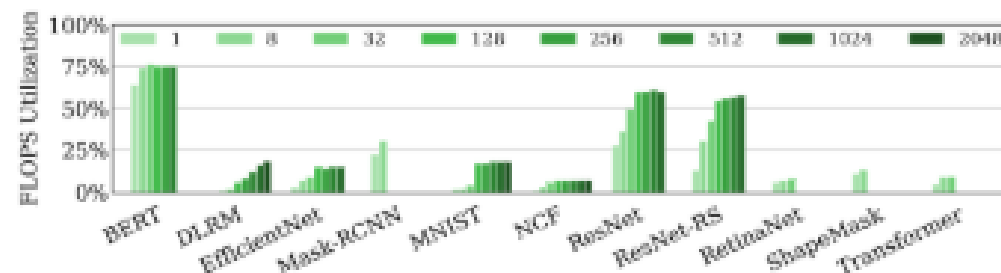
Background and Motivation

- NPU 코어 구성 요소
 1. 명령 프로세서
 2. DMA 엔진
 3. Compute 엔진
 - 3-1. SRAM
 - 3-2. Sas
 - 3-2. VUs
 4. HBM
- 클라우드에서 사용자에게 NPU를 제공하는 사실상의 표준 방식은 PCIe 패스스루
->하나의 NPU 장치를 하나의 VM 또는 컨테이너에 독점적으로 할당하는 것
- 다른 사용자가 동일한 NPU를 공유하는 것 방지
but 완전히 활용할 수 없는 경우 하드웨어 저활용으로 이어짐

Background and Motivation

- FLOPS: 컴퓨팅 리소스 활용도
- 대부분의 ML 워크로드는 TPU 코어 총 FLOPS의 절반 미만을 활용
- 주요 원인: TPU에서 가장 많은 FLOPS를 제공하는 SA가 시간적으로 저활용됨

DNN Model Names	Category
BERT, Transformer	Natural Language Processing
DLRM, NCF	Recommendation
Mask-RCNN, RetinaNet, ShapeMask	Object Detection & Segmentation
MNIST, ResNet, ResNet-RS, EfficientNet	Image Classification



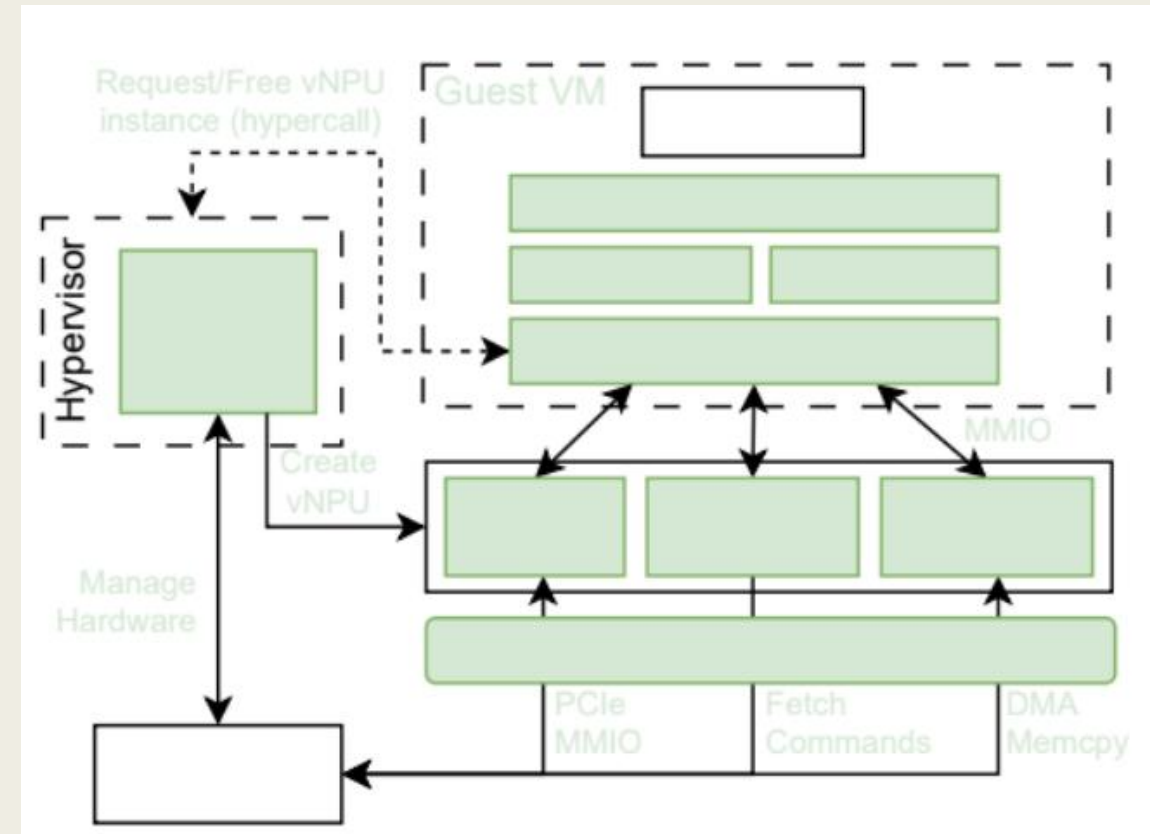
Background and Motivation

- ML 워크로드의 성능을 확장하는 일반적인 방법은 NPU 코어 내의 SA 및 VU 수를 늘리는 것 but 모든 워크로드가 이점을 얻지는 못함 -> 자원 낭비로 이어짐
- 워크로드에 할당되는 SA와 VU의 수는 수요에 따라 유연해야 함

Towards NPU Virtualization

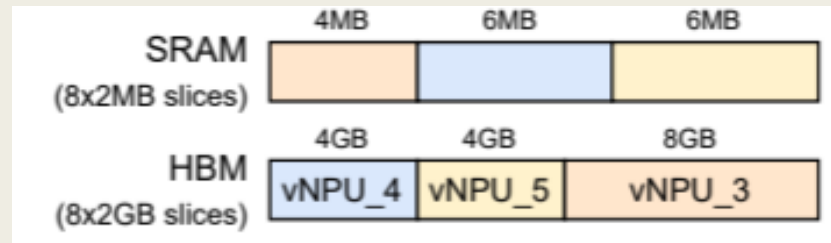
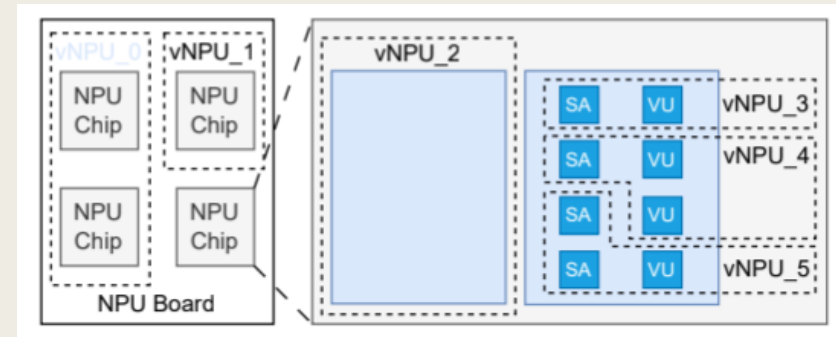
Towards NPU Virtualization

- NPU 코어 가상화에 필수적인 지원
-> 실행 유닛 단위로 코어 분할
- MIMD(다중 명령 다중 데이터) 컴퓨팅 패러다임 따름
- 하드웨어 기반 메모리 주소 변환
-> vNPU 간의 메모리 격리 가능



Towards NPU Virtualization

- vNPU를 통한 세밀한 자원 추상화
- DNN 워크로드의 수요에 따라 서로 다른 수의 SA와 VU 할당
- 또한 코어 내 SRAM 크기와 HBM 크기 맞춤 설정 가능



Towards NPU Virtualization

- Pcle 장치로 각 vNPU 노출시킴
- vNPU 계층 구조 조회 가능

```
struct vNPU_Board {  
    size_t  num_chips;  
    size_t  num_cores_per_chip;  
    size_t  num_SAs_per_core;  
    size_t  num_VUs_per_core;  
    size_t  sram_size_per_core;  
    size_t  mem_size_per_core;  
}
```


Towards NPU Virtualization

- vNPU 생명 주기
 1. 필요한 NPU 코어 수와 크기 지정
 2. vNPU 초기화 시 반가상화 인터페이스를 통해 하이퍼바이저에 요청 보냄
 3. vNPU 메니저는 할당 가능한 NPU 하드웨어 자원 식별, 게스트 VM이 vNPU에 접근할 수 있도록 MMIO 매핑 생성
 4. 커맨드 버퍼를 통해 연산 오프로딩 명령어 내림/IOMMU를 통해 DMA 접근 권한 가짐
 5. 사용자가 vNPU 인스턴스를 분리 및 해제하거나 다른 vNPU 구성 요청

Towards NPU Virtualization

- 할당기는 vNPU 인스턴스에 SA, VU, SRAM/HBM 적절한 조합 할당
- x: SA 활성 실행 시간
y: VU 활성 실행 시간
S: 예상 가속도
U: nx개의 SA와 ny개의 VU에서 워크로드 실행했을 때의 활용도
k: nx/ny
- 전체 EU 수량은 게스트 VM에서 실행되는 ML 워크로드의 SLO를 만족하는 가장 작은 값으로 결정됨
- 모델별로 최적의 SA와 VU 비율을 계산하여 압달의 법칙에 의한 성능 한계를 극복하고 자원 낭비를 최소화
-

$$S = (x + y) / \left(\frac{x}{n_x} + \frac{y}{n_y} \right)$$

$$U = \frac{S}{S_h} = \frac{(x + y)n_x n_y}{(x n_y + y n_x)(n_x + n_y)}$$

$$U = \frac{k(1 + a)}{(k + a)(k + 1)}, \quad k > 0, a > 0$$

Towards NPU Virtualization

- 초기 매핑(Initial mapping): 새로운 vNPU를 할당하기 위해 vNPU 매니저는 SA/VU 구성과 요청된 메모리 크기를 입력으로 받음
- EU 활용도를 최대화하기 위해 NeuCloud의 vNPU 매니저는 모든 vNPU의 총 EU 수가 NPU 코어의 가용 EU 수를 초과하지 않는 범위 내에서 vNPU들을 그룹화
- 초기 매핑 이후 워크로드 패턴이 시간이 지나면서 변할 수 있으므로 스케줄러는 vNPU를 재배치함

Towards NPU Virtualization

- NPU 코어의 서로 다른 실행 유닛(EU)에 vNPU를 할당한 후에도, 오프로드된 워크로드가 없을 때 이러한 vNPU들은 유휴 상태가 될 수 있습니다. 이는 시간적인 저활용(temporal underutilization) 야기함 -> NeuCloud는 코어에 더 많은 vNPU를 매핑(과다 할당)
- 2가지 경우
 1. NeuCloud는 동일한 SA/VU 구성을 가진 vNPU들만 동일한 NPU 코어에 할당
 2. vNPU들이 임의의 SA/VU 구성을 가질 수 있는 경우

Towards NPU Virtualization

- vNPU 인스턴스 간의 런타임 격리 강제해야 함-> 보안 격리 + 성능 격리
- 보안 격리: 메모리 주소 공간 격리를 통해 강제
- 성능 격리: 하드웨어와 소프트웨어 기술의 협력을 통해 강제

Conclusion

Conclusion

- 본 논문에서 우리는 NPU 가상화를 실용적으로 만들기 위해 필요한 하드웨어 지원에 대해 논의함
- 세밀한 자원 할당의 필요성 뿐만 아니라, vNPU 간의 보안 및 성능 격리와 같은 NPU 가상화의 주요 도전 과제들을 확인함
- 향후 실제 작동하는 프로토타입을 제작하기 위해 계획하고 있음
+ 구축된 시스템을 실제 ML 서비스에 적용 -> 성능을 증명